

Comparison of DCS and SCOS for Cerebral Blood-Flow Sensing Using Monte Carlo Photon Transport

https://github.com/DanSadeh/MonteCarlo_HW3_DanSadeh_314631763

Dan Sadeh | ID: 314631763 | Neurophotonics Lab

1. Objective

This report analyzes pre-computed Monte Carlo photon-transport data to compare two diffuse optical methods for measuring cerebral blood flow: diffuse correlation spectroscopy (DCS) and speckle-contrast optical spectroscopy (SCOS). For each detected photon we use its statistical weight and its per-layer momentum-transfer summary to reconstruct the electric-field autocorrelation, fit a blood-flow index (BFI), and quantify three performance metrics: cerebral sensitivity, coefficient of variation (CoV), and contrast-to-noise ratio (CNR). We evaluate how these metrics depend on source-detector separation, the DCS correlation-fit window, the SCOS camera exposure time, optical wavelength (850 versus 1064 nm), and extracerebral (scalp, skull, CSF) thickness. The expected result is an optimal operating regime that maximizes the ability to detect a brain blood-flow change through the overlying tissue.

2. Theoretical Background

The analysis converts stored photon histories into a field correlation and then into each instrument's measurable observable. Each detected photon decorrelates at a rate set by how much momentum it exchanges with moving scatterers in each tissue layer, given by $\Gamma_i = 2 k_0^2 n^2 \sum_{\ell} Y_{i,\ell} \cdot \text{BFI}_{\ell}$ (Eq. 1), where $k_0 = 2\pi/\lambda$ is the vacuum wavenumber, n the tissue refractive index, $Y_{i,\ell}$ the photon's momentum transfer in layer ℓ , and BFI_{ℓ} that layer's blood-flow index. We need Eq. 1 because it links the microscopic blood flow of each layer to a single measurable decay rate per photon, which is what ultimately makes deep brain flow detectable at the surface.

Averaging the weighted photon decays gives the unnormalized field autocorrelation $G_1(\tau) = (1/N) \sum_i w_i \exp(-\Gamma_i \tau)$, normalized as $g_1(\tau) = G_1(\tau) / \bar{w}$ (Eq. 2), where w_i is the photon weight and $\bar{w}$ the mean weight. We need Eq. 2 because $g_1(\tau)$ is the common physical quantity from which both the DCS and SCOS observables are derived.

DCS measures the intensity autocorrelation, related to the field correlation through the Siegert relation $g_2(\tau) = 1 + \beta |g_1(\tau)|^2$ (Eq. 3), where β is the coherence factor. We need Eq. 3 because the DCS detector records intensity fluctuations rather than the field directly, so the model must be expressed in g_2 .

SCOS instead measures the spatial speckle contrast integrated over the camera exposure time T , given by $K^2(T) = (2\beta / T) \int_0^T |g_1(\tau)|^2 (1 - \tau/T) d\tau$ (Eq. 4). We need Eq. 4 because a camera with finite exposure blurs the speckle pattern by an amount that depends on how fast g_1 decays, which converts blood flow into a measurable loss of speckle contrast.

3. Methodology and Results

We did not run any photon simulation; the four-layer transport data were loaded from the provided Data_Files folder, grouped by source-detector separation, and analyzed by the completed analysis function. For every geometry and wavelength we computed $g_1(\tau)$ from Eqs. 1 and 2, formed the DCS and SCOS observables from Eqs. 3 and 4, fit the BFI with a bounded log-space inverse search, and estimated CoV and CNR from repeated noisy refits at a 10 Hz sampling rate. All figures were exported from MATLAB at 600 DPI.

3.1 Head geometry and measurement configuration

The forward model is a planar four-layer head consisting of scalp (5 mm), skull (3 to 13 mm), CSF (2 mm), and brain (40 mm), with a single surface source and a line of detectors spaced 5 to 40 mm away. The three panels share the scalp, CSF, and brain layers but thicken the skull so that the total extracerebral thickness is 10, 15, and 20 mm; this is the central independent variable, because a thicker low-information layer pushes the brain deeper and forces light to travel farther before it can sample cerebral flow.

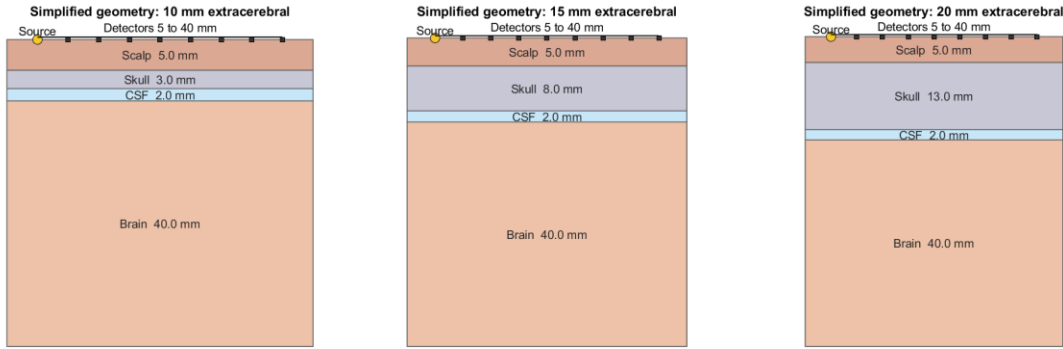


Figure 1: Simplified four-layer head geometry for the three extracerebral thicknesses, with the surface source and the 5 to 40 mm detector line.

3.2 DCS: effect of the correlation fit window

Figure 2 shows the three DCS metrics versus the fraction of the g_2 decay curve used for the BFI fit, with one curve per source-detector separation. Panel (a) shows cerebral sensitivity rising strongly with separation, from near 0 percent at 5 mm to about 72 percent at 40 mm, because wider separations send photons along deeper banana-shaped paths that accumulate more brain momentum transfer, increasing the brain-layer term of Eq. 1. Within each curve, sensitivity falls modestly as more of the slow decay tail is included (the 40 mm curve drops from about 72 to 58 percent before levelling near 60), indicating that the early, fast-decaying part of g_2 carries the most brain-specific information. Panel (b) shows the coefficient of variation: noise grows by orders of magnitude with separation because few photons reach distant detectors and the count rate falls, while within each curve the noise decreases as more of the decay is fit because additional points stabilize the inverse fit. Panel (c) shows the contrast-to-noise ratio, the ratio of the previous two quantities: it is maximized at an intermediate separation of about 20 mm, with the 15 and 25 mm curves close behind (15 mm leads at small fit fractions and 25 mm overtakes it once most of the decay is included), because at mid separations sensitivity is already substantial but photon starvation has not yet dominated. The practical lesson is that DCS should use a mid-range separation near 20 mm and fit most of the decay curve.

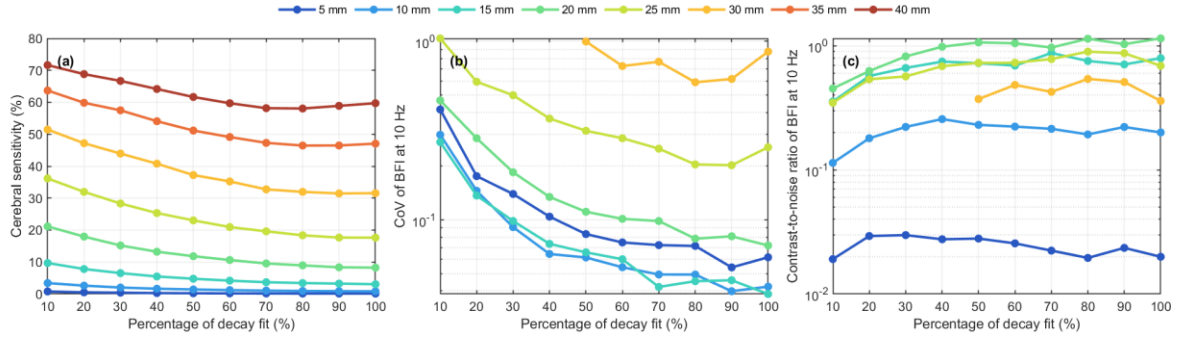


Figure 2: DCS cerebral sensitivity (a), coefficient of variation (b), and contrast-to-noise ratio (c) at 10 Hz versus the percentage of the decay curve used for the BFI fit, for source-detector separations from 5 to 40 mm.

3.3 SCOS: effect of camera exposure time

Figure 3 shows the same three metrics for SCOS versus camera exposure time, again with one curve per separation. Panel (a) shows that cerebral sensitivity is high at very short exposures, starting near 73 percent for the largest separation, then decreasing and settling onto a separation-dependent plateau of about 44 percent beyond roughly 10^{-4} s; once the exposure is long enough to integrate the full g_1 decay in Eq. 4, sensitivity no longer depends on exposure and is fixed by geometry and separation. Panel (b) shows a characteristic U-shaped coefficient of variation: at short exposure too few photoelectrons are collected per frame so shot noise dominates, whereas at long exposure fewer independent frames are averaged within the 10 Hz sampling window, so an intermediate exposure of roughly 10^{-3} to 10^{-2} s minimizes the noise. Panel (c) shows that the contrast-to-noise ratio therefore peaks at an intermediate exposure near 10^{-3} s, with the highest values, on the order of 150, achieved by the 20 and 25 mm separations. The practical lesson is that SCOS has a clear optimal exposure that trades photon collection against temporal averaging, again favouring a mid-range separation.

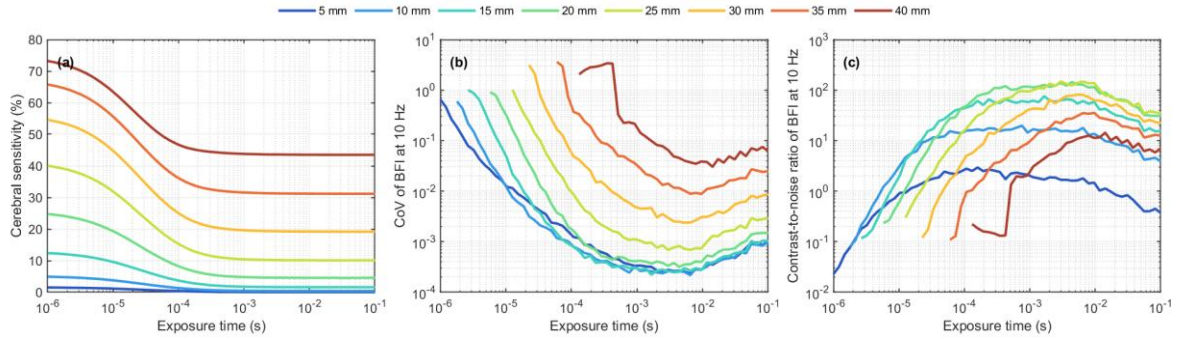


Figure 3: SCOS cerebral sensitivity (a), coefficient of variation (b), and contrast-to-noise ratio (c) at 10 Hz versus camera exposure time, for source-detector separations from 5 to 40 mm.

3.4 Modality and wavelength comparison across geometries

Figure 8 compares 850 nm DCS, 1064 nm DCS, and 850 nm SCOS as a function of separation, with rows for the three extracerebral thicknesses; SCOS was evaluated only at 850 nm, while 1064 nm was analyzed

in DCS mode only. The sensitivity column (a, d, g) shows that all three modalities behave similarly and rise with separation, but the absolute sensitivity collapses as the extracerebral layer thickens, falling from about 75 percent at 40 mm for the 10 mm geometry to only about 20 percent for the 20 mm geometry, confirming that depth sensitivity is governed primarily by geometry. The CoV column (b, e, h) reveals the decisive difference: SCOS noise is one to three orders of magnitude below DCS at every separation because the camera samples many speckles in parallel, and among the DCS curves the two wavelengths nearly overlap out to about 15 mm separation, beyond which 1064 nm is consistently quieter than 850 nm owing to its higher detected count rate; the DCS curves also terminate early at large separations where noisy fits collapse onto the solver bounds and are rejected. The CNR column (c, f, i) follows directly and shows the strongest result of the study. For the 10 mm geometry the peak contrast-to-noise reaches about 500 for SCOS, about 10 for 1064 nm DCS, and about 4 for 850 nm DCS; for the 15 mm geometry it falls to about 105, 3, and 1 respectively; and for the 20 mm geometry it drops further to about 11, 0.9, and 0.1. In every row SCOS dominates by one to two orders of magnitude, 1064 nm DCS outperforms 850 nm DCS, and every curve peaks near a 20 mm separation. The thicker the extracerebral tissue, the lower the entire CNR family becomes, an approximately fifty-fold collapse between the 10 and 20 mm geometries. The practical lesson is that, despite slightly lower raw sensitivity, SCOS is the superior modality because its parallel detection overwhelmingly reduces noise, a longer wavelength further benefits DCS, and skull thickness is the dominant limit on performance.

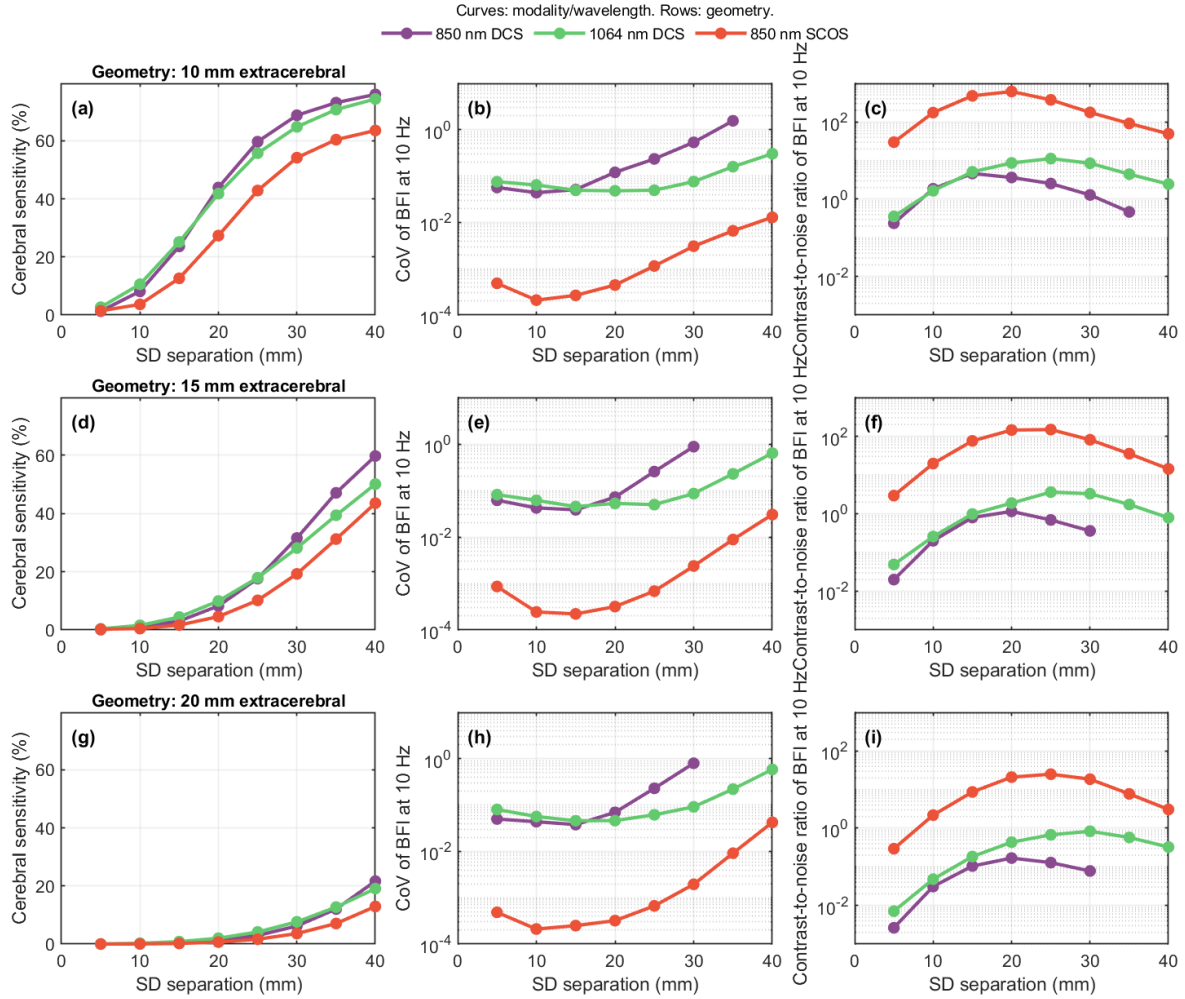


Figure 8: Cerebral sensitivity (left column), coefficient of variation (center), and contrast-to-noise ratio (right) at 10 Hz versus source-detector separation, comparing 850 nm DCS, 1064 nm DCS, and 850 nm SCOS, with rows for 10, 15, and 20 mm extracerebral thickness. Exported at 600 DPI.

3.5 Conclusions

Across all geometries the results converge on a consistent picture: cerebral blood-flow sensing is limited not by sensitivity but by noise. Sensitivity is set mainly by source-detector separation and extracerebral thickness, both of which are geometric, whereas the achievable contrast-to-noise ratio is dominated by how efficiently each instrument collects and averages photons. SCOS, by reading many speckles simultaneously, attains the lowest coefficient of variation and therefore by far the highest contrast-to-noise ratio, peaking near a 20 mm separation; DCS benefits from the longer 1064 nm wavelength and from fitting most of its decay curve, and also peaks near 20 mm. Increasing the extracerebral thickness from 10 to 20 mm reduces the peak contrast-to-noise ratio roughly fifty-fold for every modality, which quantifies the central difficulty of non-invasive brain monitoring and motivates the careful joint optimization of separation, exposure, and wavelength.

Code Appendix

```
% Dan Sadeh 314631763

function results =
MC_Neurophotonic_Assignment_Analysis_314631763_Dan_Sadeh(cfg, transport)
%% GPU PMC Neurophotonic Assignment - Analysis Version
% This file converts stored photon-history summaries into DCS and SCOS
% observables. The DCS/SCOS decorrelation equation uses the stored Y values,
% which are per-layer momentum-transfer summaries, not raw path length.
%
% Teaching organization:
%   1. Start the result tree.
%   2. Compute DCS metrics directly in the main reading path.
%   3. Compute SCOS metrics directly in the main reading path.
%   4. Keep only equation-level helpers below the main analysis.
%
% Student work areas are boxed with STUDENT FILL-IN BLOCK.
% Fill only the blank lines in those blocks. DCS fitting is provided.
%
% MATLAB note:
%   Lines that start with % are comments and will not run.

%% 1. Start the result tree
% Purpose:
%   Collect every analysis output in one result struct while keeping the
%   important setup variables visible.
%
% Variables to notice:
%   cfg          - simulation and analysis settings
%   transport    - detected photon weights and path summaries
%   grouped      - transport grouped by detector separation

fprintf('\n-----\n');
fprintf('Analyzing stored unpolarized photon transport\n');
fprintf('-----\n');

grouped = transport.grouped;
reflectance_per_detector = transport.reflectance_per_detector(:);

results = struct();
results.cfg_snapshot = cfg;
results.detector_rho_mm = cfg.detector_rho_mm(:);
results.reflectance_per_detector = reflectance_per_detector;

baseline_brain_bfi = double(cfg.BFi_baseline_cm2_s(cfg.brain_layer_index));
perturbed_brain_bfi = double(cfg.BFi_perturbed_cm2_s(cfg.brain_layer_index));
results.flow_change_fraction = (perturbed_brain_bfi - baseline_brain_bfi) /
max(baseline_brain_bfi, eps);

reference_index = find(abs(cfg.detector_rho_mm(:) -
cfg.detector_reference_rho_mm) < 1e-6, 1);
if isempty(reference_index)
    error('Reference detector separation %.1f mm was not found in
cfg.detector_rho_mm.', cfg.detector_reference_rho_mm);
end
```

```

reference_reflectance = max(reflectance_per_detector(reference_index), eps);
relative_reflectance = reflectance_per_detector / reference_reflectance;

%% 2. DCS analysis
% Purpose:
%   Build DCS correlation curves, fit BFI from selected fitting windows, and
%   estimate sensitivity, CoV, and CNR.
%
% Variables to notice:
%   tau           - DCS correlation delay axis
%   g1_baseline   - electric field correlation at baseline flow
%   g2_baseline   - intensity correlation measured by DCS
%   fit_fraction  - fraction of the decay curve used for one BFI fit

if strcmpi(cfg.mode, 'DCS_ONLY') || strcmpi(cfg.mode, 'BOTH')
    fprintf('\nComputing DCS metrics\n');

    nd = numel(grouped);
    nf = numel(cfg.dcs_fit_fractions);
    tau = cfg.tau_s(:);

    results.DCS = struct();
    results.DCS.tau_s = tau;
    results.DCS.fit_fractions = cfg.dcs_fit_fractions;
    results.DCS.relative_reflectance = relative_reflectance;
    results.DCS.baseline_g1 = zeros(numel(tau), nd);
    results.DCS.perturbed_g1 = zeros(numel(tau), nd);
    results.DCS.baseline_g2 = zeros(numel(tau), nd);
    results.DCS.perturbed_g2 = zeros(numel(tau), nd);
    results.DCS.sensitivity = zeros(nd, nf);
    results.DCS.cov = zeros(nd, nf);
    results.DCS.cnr = zeros(nd, nf);
    results.DCS.fit_bfi_baseline = zeros(nd, nf);
    results.DCS.fit_bfi_perturbed = zeros(nd, nf);
    results.DCS.count_rate_cps = zeros(nd, 1);

    for detector_index = 1:nd
        fprintf('   DCS at separation %2d mm\n',
            round(grouped(detector_index).rho_mm));

        [~, g1_baseline] = photon_g1_curve(cfg, grouped(detector_index), tau,
            cfg.BFi_baseline_cm2_s);
        [~, g1_perturbed] = photon_g1_curve(cfg, grouped(detector_index),
            tau, cfg.BFi_perturbed_cm2_s);

        %% ===== PROVIDED DCS CODE: SIEGERT RELATION =====
        g2_baseline = 1 + cfg.beta_dcs * abs(g1_baseline).^2;
        g2_perturbed = 1 + cfg.beta_dcs * abs(g1_perturbed).^2;
        %% ===== END PROVIDED DCS CODE =====

        results.DCS.baseline_g1(:, detector_index) = g1_baseline;
        results.DCS.perturbed_g1(:, detector_index) = g1_perturbed;
        results.DCS.baseline_g2(:, detector_index) = g2_baseline;
        results.DCS.perturbed_g2(:, detector_index) = g2_perturbed;
    end
end

```

```

        count_rate_cps = cfg.dcs_modes *
cfg.dcs_count_rate_per_mode_at_25mm_cps *
relative_reflectance(detector_index) + cfg.dcs_dark_count_cps;
        count_rate_cps = max(count_rate_cps, 1);
        results.DCS.count_rate_cps(detector_index) = count_rate_cps;

        g2_signal = max((g2_baseline(:) - 1) / max(cfg.beta_dcs, eps), 1e-
12);
        valid_g2_signal = isfinite(g2_signal) & g2_signal > 0;
        fit_coefficients = polyfit(tau(valid_g2_signal),
log(g2_signal(valid_g2_signal)), 1);
        gamma_est = max(-0.5 * fit_coefficients(1), 1);
        sigma_tau = dcs_sigma_tau(cfg, tau, gamma_est, count_rate_cps);

        for fraction_index = 1:nf
            fit_fraction = cfg.dcs_fit_fractions(fraction_index);
            dcs_decay_signal = g2_baseline(:) - 1;
            signal_start = max(dcs_decay_signal(1), eps);
            target_value = signal_start * max(1 - fit_fraction, 0);
            last_fit_index = find(dcs_decay_signal <= target_value, 1);
            if isempty(last_fit_index)
                last_fit_index = numel(dcs_decay_signal);
            end
            fit_mask = false(size(dcs_decay_signal));
            fit_mask(1:last_fit_index) = true;
            tau_fit = tau(fit_mask);
            g2_baseline_fit = g2_baseline(fit_mask);
            g2_perturbed_fit = g2_perturbed(fit_mask);

            %% ===== PROVIDED DCS CODE: BFI FIT =====
            bfi_baseline = solve_bfi_in_log_space(@(trial_bfi) sum((1 +
cfg.beta_dcs * abs(semi_infinite_g1(cfg, grouped(detector_index).rho_mm,
tau_fit, trial_bfi)).^2 - g2_baseline_fit) .^ 2), cfg);
            bfi_perturbed = solve_bfi_in_log_space(@(trial_bfi) sum((1 +
cfg.beta_dcs * abs(semi_infinite_g1(cfg, grouped(detector_index).rho_mm,
tau_fit, trial_bfi)).^2 - g2_perturbed_fit) .^ 2), cfg);
            results.DCS.fit_bfi_baseline(detector_index, fraction_index) =
bfi_baseline;
            results.DCS.fit_bfi_perturbed(detector_index, fraction_index) =
bfi_perturbed;
            results.DCS.sensitivity(detector_index, fraction_index) =
((bfi_perturbed - bfi_baseline) / max(bfi_baseline, eps)) /
max(results.flow_change_fraction, eps);
            %% ===== END PROVIDED DCS CODE =====

            noisy_fits = zeros(cfg.dcs_realizations, 1);
            for realization_index = 1:cfg.dcs_realizations
                noisy_g2 = g2_baseline + sigma_tau .*
randn(size(g2_baseline));
                noisy_g2 = max(noisy_g2, 1 + 1e-12);
                noisy_g2_fit = noisy_g2(fit_mask);
                %% ===== PROVIDED DCS CODE: NOISY REFIT =====
                noisy_fits(realization_index) =
solve_bfi_in_log_space(@(trial_bfi) sum((1 + cfg.beta_dcs *
abs(semi_infinite_g1(cfg, grouped(detector_index).rho_mm, tau_fit,
trial_bfi)).^2 - noisy_g2_fit) .^ 2), cfg);
                %% ===== END PROVIDED DCS CODE =====

```



```

        end

        results.DCS.cov(detector_index, fraction_index) =
coefficient_of_variation_from_noisy_fits(noisy_fits, cfg);
        if ~isfinite(results.DCS.cov(detector_index, fraction_index)) ||
results.DCS.cov(detector_index, fraction_index) <= 0
            results.DCS.cnr(detector_index, fraction_index) = NaN;
        else
            results.DCS.cnr(detector_index, fraction_index) =
results.DCS.sensitivity(detector_index, fraction_index) /
results.DCS.cov(detector_index, fraction_index);
        end
    end
end

end

%% 3. SCOS analysis
% Purpose:
%   Integrate field correlations over camera exposure time, fit BFI from
%   speckle contrast, and estimate sensitivity, CoV, and CNR.
%
% Variables to notice:
%   tau_scos      - delay axis used for the exposure integral
%   exposure_s    - one camera exposure time
%   sp_ratio      - speckle-to-pixel ratio used in the noise model
%   kf2_baseline  - baseline fundamental speckle contrast squared

if strcmpi(cfg.mode, 'SCOS_ONLY') || strcmpi(cfg.mode, 'BOTH')
    fprintf('\nComputing SCOS metrics\n');

    nd = numel(grouped);
    ne = numel(cfg.exposure_s);
    ns = numel(cfg.scos_sp_ratios);
    tau_scos = cfg.tau_scos_s(:);
    texp = cfg.exposure_s(:);

    results.SCOS = struct();
    results.SCOS.exposure_s = texp;
    results.SCOS.sp_ratios = cfg.scos_sp_ratios;
    results.SCOS.relative_reflectance = relative_reflectance;
    results.SCOS.baseline_kf2 = zeros(ne, nd);
    results.SCOS.perturbed_kf2 = zeros(ne, nd);
    results.SCOS.fit_bfi_baseline = zeros(nd, ne);
    results.SCOS.fit_bfi_perturbed = zeros(nd, ne);
    results.SCOS.sensitivity = zeros(nd, ne);
    results.SCOS.cov = zeros(nd, ne);
    results.SCOS.cnr = zeros(nd, ne);
    results.SCOS.best_exposure_idx = zeros(nd, 1);
    results.SCOS.best_exposure_s = zeros(nd, 1);
    results.SCOS.best_sensitivity = zeros(nd, 1);
    results.SCOS.best_cov = zeros(nd, 1);
    results.SCOS.best_cnr = zeros(nd, 1);
    results.SCOS.sp_ratio_cnr_15mm = zeros(ns, 1);
    results.SCOS.sp_ratio_cnr_30mm = zeros(ns, 1);

    baseline_g1_cache = cell(nd, 1);
    perturbed_g1_cache = cell(nd, 1);

```

```

    for detector_index = 1:nd
        fprintf(' SCOS at separation %2d mm\n',
round(grouped(detector_index).rho_mm));

        [~, baseline_g1_cache{detector_index}] = photon_g1_curve(cfg,
grouped(detector_index), tau_scos, cfg.BFi_baseline_cm2_s);
        [~, perturbed_g1_cache{detector_index}] = photon_g1_curve(cfg,
grouped(detector_index), tau_scos, cfg.BFi_perturbed_cm2_s);

        for exposure_index = 1:ne
            exposure_s = texp(exposure_index);
            sp_ratio = cfg.scos_default_sp_ratio;

            kf2_baseline = scos_kf2_from_g1(tau_scos,
baseline_g1_cache{detector_index}, exposure_s, cfg.beta_scos);
            kf2_perturbed = scos_kf2_from_g1(tau_scos,
perturbed_g1_cache{detector_index}, exposure_s, cfg.beta_scos);
            bfi_baseline = fit_bfi_from_scos(cfg,
grouped(detector_index).rho_mm, exposure_s, kf2_baseline, cfg.beta_scos);
            bfi_perturbed = fit_bfi_from_scos(cfg,
grouped(detector_index).rho_mm, exposure_s, kf2_perturbed, cfg.beta_scos);

            %% ===== STUDENT FILL-IN BLOCK SCOS-1A
            =====
            sensitivity = ((bfi_perturbed - bfi_baseline) / max(bfi_baseline,
eps)) / max(results.flow_change_fraction, eps);
            %% ===== END STUDENT FILL-IN BLOCK SCOS-1A
            =====

            noise = scos_noise_model(cfg,
relative_reflectance(detector_index), exposure_s, sp_ratio, kf2_baseline);
            noisy_fits = zeros(cfg.scos_realizations, 1);

            for realization_index = 1:cfg.scos_realizations
                noisy_kf2 = max(kf2_baseline + noise.sigma_kf2 * randn(), 1e-
12);
                noisy_fits(realization_index) = fit_bfi_from_scos(cfg,
grouped(detector_index).rho_mm, exposure_s, noisy_kf2, cfg.beta_scos);
            end

            cov_value = coefficient_of_variation_from_noisy_fits(noisy_fits,
cfg);

            results.SCOS.baseline_kf2(exposure_index, detector_index) =
kf2_baseline;
            results.SCOS.perturbed_kf2(exposure_index, detector_index) =
kf2_perturbed;
            results.SCOS.fit_bfi_baseline(detector_index, exposure_index) =
bfi_baseline;
            results.SCOS.fit_bfi_perturbed(detector_index, exposure_index) =
bfi_perturbed;
            results.SCOS.sensitivity(detector_index, exposure_index) =
sensitivity;
            results.SCOS.cov(detector_index, exposure_index) = cov_value;
            if ~isfinite(cov_value) || cov_value <= 0
                results.SCOS.cnr(detector_index, exposure_index) = NaN;
            end
        end
    end

```

```

        else
            results.SCOS.cnr(detector_index, exposure_index) =
sensitivity / cov_value;
        end
    end

    [results.SCOS.best_cnr(detector_index),
results.SCOS.best_exposure_idx(detector_index)] =
max(results.SCOS.cnr(detector_index, :));
    results.SCOS.best_exposure_s(detector_index) =
texp(results.SCOS.best_exposure_idx(detector_index));
    results.SCOS.best_sensitivity(detector_index) =
results.SCOS.sensitivity(detector_index,
results.SCOS.best_exposure_idx(detector_index));
    results.SCOS.best_cov(detector_index) =
results.SCOS.cov(detector_index,
results.SCOS.best_exposure_idx(detector_index));
    end

    idx_15 = find(abs(cfg.detector_rho_mm - 15) < 1e-6, 1);
    idx_30 = find(abs(cfg.detector_rho_mm - 30) < 1e-6, 1);

    if ~isempty(idx_15)
        for sp_index = 1:ns
            best_cnr = -inf;
            for exposure_index = 1:ne
                exposure_s = texp(exposure_index);
                sp_ratio = cfg.scos_sp_ratios(sp_index);
                kf2_baseline = scos_kf2_from_g1(tau_scos,
baseline_g1_cache{idx_15}, exposure_s, cfg.beta_scos);
                kf2_perturbed = scos_kf2_from_g1(tau_scos,
perturbed_g1_cache{idx_15}, exposure_s, cfg.beta_scos);
                bfi_baseline = fit_bfi_from_scos(cfg, grouped(idx_15).rho_mm,
exposure_s, kf2_baseline, cfg.beta_scos);
                bfi_perturbed = fit_bfi_from_scos(cfg,
grouped(idx_15).rho_mm, exposure_s, kf2_perturbed, cfg.beta_scos);
                %% ===== STUDENT FILL-IN BLOCK SCOS-1B
=====
                sensitivity = ((bfi_perturbed - bfi_baseline) /
max(bfi_baseline, eps)) / max(results.flow_change_fraction, eps);
                %% ===== END STUDENT FILL-IN BLOCK SCOS-1B
=====
                noise = scos_noise_model(cfg, relative_reflectance(idx_15),
exposure_s, sp_ratio, kf2_baseline);
                noisy_fits = zeros(cfg.scos_realizations, 1);
                for realization_index = 1:cfg.scos_realizations
                    noisy_kf2 = max(kf2_baseline + noise.sigma_kf2 * randn(),
1e-12);
                    noisy_fits(realization_index) = fit_bfi_from_scos(cfg,
grouped(idx_15).rho_mm, exposure_s, noisy_kf2, cfg.beta_scos);
                end
                cov_value =
coefficient_of_variation_from_noisy_fits(noisy_fits, cfg);
                if isfinite(cov_value) && cov_value > 0
                    best_cnr = max(best_cnr, sensitivity / cov_value);
                end
            end
        end
    end
end

```

```

        results.SCOS.sp_ratio_cnr_15mm(sp_index) = best_cnr;
    end
end

if ~isempty(idx_30)
    for sp_index = 1:ns
        best_cnr = -inf;
        for exposure_index = 1:ne
            exposure_s = texp(exposure_index);
            sp_ratio = cfg.scos_sp_ratios(sp_index);
            kf2_baseline = scos_kf2_from_g1(tau_scos,
baseline_g1_cache{idx_30}, exposure_s, cfg.beta_scos);
            kf2_perturbed = scos_kf2_from_g1(tau_scos,
perturbed_g1_cache{idx_30}, exposure_s, cfg.beta_scos);
            bfi_baseline = fit_bfi_from_scos(cfg, grouped(idx_30).rho_mm,
exposure_s, kf2_baseline, cfg.beta_scos);
            bfi_perturbed = fit_bfi_from_scos(cfg,
grouped(idx_30).rho_mm, exposure_s, kf2_perturbed, cfg.beta_scos);
            %% ===== STUDENT FILL-IN BLOCK SCOS-1C
            =====
            sensitivity = ((bfi_perturbed - bfi_baseline) /
max(bfi_baseline, eps)) / max(results.flow_change_fraction, eps);
            %% ===== END STUDENT FILL-IN BLOCK SCOS-1C
            =====
            noise = scos_noise_model(cfg, relative_reflectance(idx_30),
exposure_s, sp_ratio, kf2_baseline);
            noisy_fits = zeros(cfg.scos_realizations, 1);
            for realization_index = 1:cfg.scos_realizations
                noisy_kf2 = max(kf2_baseline + noise.sigma_kf2 * randn(),
1e-12);
                noisy_fits(realization_index) = fit_bfi_from_scos(cfg,
grouped(idx_30).rho_mm, exposure_s, noisy_kf2, cfg.beta_scos);
            end
            cov_value =
coefficient_of_variation_from_noisy_fits(noisy_fits, cfg);
            if isfinite(cov_value) && cov_value > 0
                best_cnr = max(best_cnr, sensitivity / cov_value);
            end
        end
        results.SCOS.sp_ratio_cnr_30mm(sp_index) = best_cnr;
    end
end
end
end

%% Helper: Photon data to field correlation
% Purpose:
%   Convert detected photon weights and per-layer momentum transfer into the
%   DCS/SCOS field correlation g1(tau).
% Inputs:
%   cfg, one detector group, delay axis tau, and layer BFI values.
% Outputs:
%   G1 is the unnormalized field correlation; g1 is normalized by zero delay.
% Variables to notice:
%   w, Y, BFi_vector, dynamic_rate.

```

```

function [G1, g1] = photon_g1_curve(cfg, group, tau, BFi_cm2_s) % Compute the
unnormalized and normalized field correlation curves from the stored photon
data.
if group.n < 1 % Return safe defaults when a detector received no photons.
    G1 = zeros(size(tau)); % Return a zero unnormalized field correlation
    curve.
    g1 = zeros(size(tau)); % Return a zero normalized field correlation
    curve.
    return; % Stop because there is nothing to evaluate.
end % Finish the empty detector guard.
%% Hint: investigate the variable group and analyze what needs to be 1D and
units
%% ===== STUDENT FILL-IN BLOCK SHARED-1 =====
w          = group.w(:);          % column vector of weights
Y          = group.Y;            % [n_photons x n_layers]
BFi_vector = BFi_cm2_s(:);       % [n_layers x 1]
k0_vac_cm_inv = 2*pi / (cfg.lambda_nm * 1e-7); % lambda_nm -> cm, then
2pi/lambda
dynamic_rate = 2 * k0_vac_cm_inv^2 * double(cfg.n_tissue)^2 * (Y *
BFi_vector); % [n_photons x 1], Gamma_i
G1          = weighted_exponential_curve(tau, dynamic_rate, w, 50000); %
G1(tau)=(1/N)sum_i w_i exp(-Gamma_i tau)

g1          = G1 / max(mean(w), eps); % normalize by mean weight
(G1(0)=mean w)
%% ===== END STUDENT FILL-IN BLOCK SHARED-1 =====
end % Finish the g1 evaluator.

%% Helper: Memory-safe photon average
% Purpose:
%   Average many photon exponential decays without building a huge matrix.
% Inputs:
%   Delay axis tau, photon decay rates, photon weights, and chunk size.
% Output:
%   G1, the weighted Monte Carlo estimate of field correlation.
function G1 = weighted_exponential_curve(tau, dynamic_rate, weights,
chunk_size)
tau = tau(:);
dynamic_rate = dynamic_rate(:);
weights = weights(:);
G1 = zeros(numel(tau), 1);
n_photons = numel(dynamic_rate);

for start_index = 1:chunk_size:n_photons
    stop_index = min(start_index + chunk_size - 1, n_photons);
    photon_slice = start_index:stop_index;
    decay_block = exp(-tau * dynamic_rate(photon_slice).'); % hint look at
the dimentions
    G1 = G1 + decay_block * weights(photon_slice); %hint look at equation 12
end

G1 = G1 / max(n_photons, 1);
end

%% Helper: Semi-infinite field correlation
% Purpose:
%   Evaluate the analytical DCS field correlation used by the inverse fits.

```

```

% Inputs:
%   cfg, source-detector separation, delay axis, and trial BFI.
% Outputs:
%   g1, normalized field correlation from the semi-infinite model.
% Variables to notice:
%   K_cm_inv, r1_cm, rb_cm, numerator, denominator.
function g1 = semi_infinite_g1(cfg, rho_mm, tau, bfi) % Evaluate the semi
infinite DCS field correlation model used by the paper fits.
mua_cm_inv = cfg.dcs_fit_mua_cm_inv; % Read the fitted absorption
coefficient.
musp_cm_inv = cfg.dcs_fit_musp_cm_inv; % Read the fitted reduced scattering
coefficient.
rho_cm = rho_mm / 10; % Convert the source detector separation from
millimeters to centimeters.
n_tissue = double(cfg.n_tissue); % Read the tissue refractive index as double
precision.
k0_vac_cm_inv = 2 * pi / (cfg.lambda_nm * 1e-7); % Convert the vacuum
wavelength into the vacuum wavenumber.
l_star_cm = 1 / musp_cm_inv; % Convert the reduced scattering coefficient
into the transport mean free path.
Reff = -1.440 * n_tissue ^ -2 + 0.710 * n_tissue ^ -1 + 0.668 + 0.0636 *
n_tissue; % Compute the diffuse boundary reflection parameter.
zb_cm = (2 / (3 * musp_cm_inv)) * ((1 + Reff) / (1 - Reff)); % Compute the
extrapolated zero boundary depth.
r1_cm = sqrt(rho_cm ^ 2 + l_star_cm ^ 2); % Compute the source image distance
for the direct source.
rb_cm = sqrt(rho_cm ^ 2 + (l_star_cm + 2 * zb_cm) ^ 2); % Compute the source
image distance for the image source.
K0_cm_inv = sqrt(3 * mua_cm_inv * musp_cm_inv); % Compute the static field
attenuation factor.
K_cm_inv = sqrt(3 * mua_cm_inv * musp_cm_inv + 6 * (k0_vac_cm_inv ^ 2) *
(n_tissue ^ 2) * (musp_cm_inv ^ 2) * bfi .* tau); % Compute the dynamic field
attenuation factor.
numerator = rb_cm .* exp(-K_cm_inv .* r1_cm) - r1_cm .* exp(-K_cm_inv .*
rb_cm); % Build the dynamic numerator of the semi infinite solution.
denominator = rb_cm * exp(-K0_cm_inv * r1_cm) - r1_cm * exp(-K0_cm_inv *
rb_cm); % Build the static denominator of the semi infinite solution.
g1 = numerator ./ max(denominator, eps); % Normalize the dynamic solution by
the zero delay field term.
end % Finish the semi infinite g1 model.

%% Helper: DCS noise model
% Purpose:
%   Estimate the standard deviation of the DCS correlation at each delay.
% Inputs:
%   cfg, delay axis, gamma estimate, and detector count rate.
% Outputs:
%   sigma_tau, the delay-dependent DCS noise level.
% Variables to notice:
%   term_one, term_two, term_three.
function sigma_tau = dcs_sigma_tau(cfg, tau, gamma_est, count_rate_cps) %
Evaluate the standard DCS correlation noise model.
bin_width = cfg.dcs_bin_width_s; % Read the correlator bin width.
sample_time = cfg.sample_time_s; % Read the 10 Hz averaging window.
nbar = max(count_rate_cps * bin_width, 1e-12); % Convert count rate into the
average counts per correlator bin.

```

```

exp_2gamma_bin = exp(-2 * gamma_est * bin_width); % Evaluate the finite bin
width decay term.
exp_2gamma_tau = exp(-2 * gamma_est * tau(:)); % Evaluate the full intensity
decay term across all delays.
exp_gamma_tau = exp(-gamma_est * tau(:)); % Evaluate the field decay term
across all delays.
term_one = cfg.beta_dcs ^ 2 .* ((1 + exp_2gamma_bin) .* (1 + exp_2gamma_tau)
+ 2 * (tau(:) / bin_width) .* (1 - exp_2gamma_bin) .* exp_2gamma_tau) ./
max(1 - exp_2gamma_bin, eps); % Compute the intrinsic correlation noise term.
term_two = 2 * (nbar ^ -1) * cfg.beta_dcs .* (1 + exp_2gamma_tau); % Compute
the mixed shot and correlation noise term.
term_three = (nbar ^ -2) .* (1 + cfg.beta_dcs .* exp_gamma_tau); % Compute
the pure shot noise term.
sigma_tau = sqrt((bin_width / sample_time) .* (term_one + term_two +
term_three)); % Combine the three terms into the delay dependent standard
deviation.
end % Finish the DCS noise model.

%% Helper: SCOS exposure integral
% Purpose:
%   Integrate g1 over exposure time to get fundamental speckle contrast.
% Inputs:
%   Delay axis tau, field correlation g1, exposure time, and beta.
% Outputs:
%   kf2, the fundamental speckle contrast squared.
% Variables to notice:
%   g1_squared, tau_use, g1_squared_use, integrand.
function kf2 = scos_kf2_from_g1(tau, g1, exposure_s, beta) % Integrate the
field correlation over the exposure time to obtain the fundamental speckle
contrast squared.
%% ===== STUDENT FILL-IN BLOCK SCOS-INTEGRAL =====
tau = tau(:);
g1_squared = abs(g1(:)).^2; % |g1(tau)|^2
sample_mask = tau < exposure_s;
tau_use = tau(sample_mask);
g1_squared_use = g1_squared(sample_mask);

if isempty(tau_use) || tau_use(1) > 0
    tau_use = [0; tau_use];
    g1_squared_use = [1; g1_squared_use];
end

if tau_use(end) < exposure_s
    endpoint_value = interp1(tau, g1_squared, exposure_s, 'linear',
'extrap'); % hint: interp1
    tau_use = [tau_use; exposure_s];
    g1_squared_use = [g1_squared_use; max(endpoint_value, 0)];
end

integrand = g1_squared_use .* (1 - tau_use / exposure_s); % |g1|^2 (1 -
tau/T)
kf2 = (2 * beta / exposure_s) * trapz(tau_use, integrand); % Kf^2(T)
%% ===== END STUDENT FILL-IN BLOCK SCOS-INTEGRAL =====
end % Finish the SCOS exposure integral.

%% Helper: SCOS BFI fit
% Purpose:

```

```

% Fit one blood flow index from one measured SCOS contrast value.
% Inputs:
%   cfg, source-detector separation, exposure time, measured kf2, and beta.
% Outputs:
%   bfi, the fitted blood flow index.
% Variables to notice:
%   The objective compares measured kf2 with a modeled exposure integral.
function bfi = fit_bfi_from_scos(cfg, rho_mm, exposure_s, measured_kf2, beta)
% Fit the SCOS inverse model for one exposure time.
bfi = solve_bfi_in_log_space(@(trial_bfi)
(scos_kf2_from_g1(cfg.tau_scos_s(:), semi_infinite_g1(cfg, rho_mm,
cfg.tau_scos_s(:), trial_bfi), exposure_s, beta) - measured_kf2) ^ 2, cfg)); %
Fit the SCOS observable in log space so the optimizer can distinguish
realistic blood flow values across orders of magnitude.
end % Finish the SCOS fitter.

%% Helper: Log-space BFI solver
% Purpose:
%   Search for BFI over orders of magnitude with a well-scaled optimizer.
% Inputs:
%   Objective function and cfg fit bounds.
% Outputs:
%   bfi, the best-fit blood flow index.
% Variables to notice:
%   log10_bfi_min, log10_bfi_max, objective_in_log10_bfi.
function bfi = solve_bfi_in_log_space(objective_in_bfi, cfg) % Solve a one-
parameter blood flow fit over a realistic logarithmic search interval.
log10_bfi_min = log10(double(cfg.bfi_fit_bounds_cm2_s(1))); % Match the lower
bound used throughout the paper style inverse fits.
log10_bfi_max = log10(double(cfg.bfi_fit_bounds_cm2_s(2))); % Match the upper
bound used throughout the paper style inverse fits.
objective_in_log10_bfi = @(trial_log10_bfi) objective_in_bfi(10 .^
trial_log10_bfi); % Reparameterize the optimizer objective so MATLAB searches
over a numerically well-scaled interval.
solver_options = optimset('Display', 'off', 'TolX', 1e-4); % Use a modest
tolerance in log10 space, which is now meaningful across the search interval.
log10_bfi = fminbnd(objective_in_log10_bfi, log10_bfi_min, log10_bfi_max,
solver_options); % Run the bounded search over the logarithm of the blood
flow index.
bfi = 10 .^ log10_bfi; % Convert the optimal log10 blood flow value back into
blood flow units.

if isempty(bfi) || ~isfinite(bfi) % Fall back to the baseline brain value
when the optimizer fails.
    bfi = double(cfg.BFi_baseline_cm2_s(cfg.brain_layer_index)); % Use the
nominal brain blood flow index as a safe default.
end % Finish the solver failure guard.
end % Finish the logarithmic blood flow solver.

%% Helper: CoV from noisy BFI fits
% Purpose:
%   Convert noisy BFI refits into coefficient of variation while rejecting
%   cases where too many fits hit the solver bounds.
% Inputs:
%   noisy_fits and cfg solver-bound settings.
% Outputs:
%   cov_value, or NaN when the noisy fit distribution is unreliable.

```



```

% Variables to notice:
%   lower_bound_hits and upper_bound_hits.
function cov_value = coefficient_of_variation_from_noisy_fits(noisy_fits,
cfg) % Convert the noisy blood flow refits into CoV while suppressing solver-
bound artifacts.
fit_min = double(cfg.bfi_fit_bounds_cm2_s(1)); % Read the lower BFI fit bound
used by the inverse solver.
fit_max = double(cfg.bfi_fit_bounds_cm2_s(2)); % Read the upper BFI fit bound
used by the inverse solver.
log10_fit_values = log10(max(double(noisy_fits(:)), realmin)); % Convert the
noisy fit values into log10 space so boundary detection can be performed
multiplicatively.
lower_bound_hits = abs(log10_fit_values - log10(fit_min)) <=
cfg.bfi_fit_boundary_log10_tolerance; % Flag noisy fits that collapsed onto
the lower solver bound.
upper_bound_hits = abs(log10_fit_values - log10(fit_max)) <=
cfg.bfi_fit_boundary_log10_tolerance; % Flag noisy fits that collapsed onto
the upper solver bound.

if mean(lower_bound_hits | upper_bound_hits) >
cfg.max_fraction_noisy_fits_on_solver_boundary % Mark the fit as unreliable
when too many noisy realizations hit either solver boundary.
    cov_value = NaN; % Return an invalid CoV so downstream plots do not
report a misleadingly optimistic CNR.
    return; % Stop because the noisy fit distribution is not trustworthy.
end % Finish the solver-bound collapse guard.

cov_value = std(noisy_fits) / max(mean(noisy_fits), eps); % Use the standard
coefficient of variation when the noisy fit distribution remains away from
the solver bounds.
end % Finish the noisy fit CoV helper.

%% Helper: SCOS noise model
% Purpose:
%   Estimate measurement noise for one SCOS exposure and speckle/pixel ratio.
% Inputs:
%   cfg, detector reflectance scaling, exposure time, sp_ratio, and kf2.
% Outputs:
%   noise struct with photon, pixel, and sigma_kf2 terms.
% Variables to notice:
%   photoelectrons_per_frame, mean_photoelectrons_per_pixel, nio_total.
function noise = scos_noise_model(cfg, relative_reflectance, exposure_s,
sp_ratio, kf2) % Estimate the corrected SCOS noise floor for one operating
point.
frame_rate_hz = min(cfg.scos_frame_rate_hz, 1 / max(exposure_s, eps)); %
Limit the effective frame rate by both the camera and the exposure time.
frames_per_sample = max(floor(frame_rate_hz * cfg.sample_time_s), 1); %
Convert the effective frame rate into the number of averaged frames per 10 Hz
sample.
filled_pixels = max(min(cfg.scos_pixels, cfg.scos_bundle_modes * sp_ratio ^
2), 1); % Estimate the number of illuminated pixels for the requested speckle
to pixel ratio.
independent_observations_per_frame = max(min(cfg.scos_bundle_modes,
cfg.scos_pixels / max(sp_ratio ^ 2, eps)), 1); % Estimate the number of
independent speckle observations per frame.

```

```

photoelectrons_per_second = cfg.scos_count_rate_per_mode_at_25mm_cps *
relative_reflectance * cfg.scos_bundle_modes * cfg.scos_qe; % Convert the
transport reflectance into detected photoelectrons per second.
photoelectrons_per_frame = max(photoelectrons_per_second * exposure_s, 1e-
12); % Convert the detected rate into photoelectrons per camera frame.
mean_photoelectrons_per_pixel = max(photoelectrons_per_frame / filled_pixels,
1e-12); % Convert the frame photoelectrons into the mean photoelectrons per
illuminated pixel.
shot_contrast_sq = 1 / mean_photoelectrons_per_pixel; % Use the standard shot
noise contribution to measured speckle contrast squared.
read_contrast_sq = (cfg.scos_read_noise_e ^ 2) /
(mean_photoelectrons_per_pixel ^ 2); % Use the standard read noise
contribution to measured speckle contrast squared.
measured_k2 = max(kf2 + shot_contrast_sq + read_contrast_sq, 1e-12); % Build
the total measured speckle contrast squared before correction.
nio_total = max(independent_observations_per_frame * frames_per_sample, 1); %
Combine frame averaging and spatial averaging into the total independent
observations.

noise = struct(); % Start the SCOS noise output struct.
noise.frame_rate_hz = frame_rate_hz; % Save the effective frame rate.
noise.frames_per_sample = frames_per_sample; % Save the number of averaged
frames per sample.
noise.filled_pixels = filled_pixels; % Save the estimated illuminated pixel
count.
noise.independent_observations_per_frame =
independent_observations_per_frame; % Save the independent observations per
frame.
noise.photoelectrons_per_frame = photoelectrons_per_frame; % Save the
detected photoelectrons per frame.
noise.mean_photoelectrons_per_pixel = mean_photoelectrons_per_pixel; % Save
the mean photoelectrons per illuminated pixel.
noise.shot_contrast_sq = shot_contrast_sq; % Save the shot noise contribution
to measured contrast squared.
noise.read_contrast_sq = read_contrast_sq; % Save the read noise contribution
to measured contrast squared.
noise.sigma_kf2 = measured_k2 * sqrt(2 / nio_total); % Approximate the
corrected fundamental contrast noise with the total measured contrast
variance scaled by the independent observations.
end % Finish the SCOS noise model.

```